

MATD10: Project 2 Report

Yuyang Wang

Chapter 1: Introduction

In this report, we present the results of our second project for the MATD10 course. In Project 1, we investigated the order of convergence, stability, and how CFL conditions affects the eight schemes mentioned. It focused on the linear advection equation

$$u_t + cu_x = 0$$

with periodic boundary conditions, where it is forced to wrap the domain around. In Project 2, it still concerns about the same PDE, but it is now equipped with the initial condition and left boundary condition:

$$\begin{cases} u(x, 0) = f(x) \\ u(a, t) = g(t) \end{cases}$$

Therefore, a major difference is that the boundaries must be treated using ghost cells and one-sided difference, which will be reflected in the [schemes.py](#). Hence, `np.roll` would not help anymore.

Chapter 2: Code Explanation

2.1 setup.py

2.1.1 grid(a, b, m)

Unlike Project 1, instead of having `np.linspace` with `endpoint=False`, this project requires the m -point spatial discretization grid, which includes both endpoints $x_0 = a$, $x_{m-1} = b$ because Project 2 no longer requires periodicity. From the document, it explicitly states that

$$x_j = a + j\Delta x, \quad j = 0, 1, \dots, m-1, \quad \Delta x = \frac{b-a}{m-1}$$

which is directly reflected from the code: `grid()` calculates and returns the desired spacing based on the input, as well as an array of all the sample grid points.

2.1.2 Other Functions in setup.py

`find_dt()` and `max_err()` are the same as in Project 1. To investigate the stability of the four schemes, it is still required to find the ratio between the maximum value and the original value, so we can just borrow it from the previous project. So is `find_dt()`, having different initial and boundary conditions does not affect the way we calculate the time step size.

Besides the two functions mentioned above, there are three additional functions in `setup.py` that are not present in Project 1: `exact_sol()`, `initial_val()`, and `left_boundary_val()`.

The function `initial_val(xs)` implements the initial condition

$$u(x, 0) = f(x) = e^{-x^2} + \cos(3x),$$

of our choice which is used to initialize the numerical solution at time $t = 0$. Given the array of grid points `xs`, the function evaluates $f(x)$ element-wise and returns the initial vector corresponding to the discrete solution at $t = 0$.

The function `left_boundary_val(t, a, c)` implements the boundary condition

$$u(a, t) = g(t) = e^{-(a-ct)^2} + \cos(3(a-ct))$$

which prescribes the value of the solution at the left boundary $x = a$ for all times $t \geq 0$. Unlike Project 1, where periodicity automatically handled boundary values, this function must be explicitly enforced at every time step when advancing the numerical solution.

The function `exact_sol(xs, t, c)` computes the exact solution

$$u(x, t) = e^{-(x-ct)^2} + \cos(3(x-ct))$$

which is known analytically for this problem. Given a set of grid points `xs` and a time t , the function evaluates the exact solution at each point. This allows us to compute the numerical error and determine the order of convergence of each scheme.

Overall, these additional functions reflect the key difference between Project 1 and Project 2: instead of relying on periodic boundary conditions, we now explicitly incorporate both the initial condition and boundary condition into the numerical method, while also using the known analytical solution to assess accuracy.

2.2 schemes.py

The main difference between the two projects is that in Project 2, we need to treat the boundaries using ghost cells and one-sided differences. Therefore, the implementation of the schemes in `schemes.py` has been modified accordingly. Though a different implementation is required for the boundary treatment, it is not difficult to translate the formulas provided by the document into code.

2.2.1 lax_friedrichs_ghost()

The Lax-Friedrichs scheme with ghost cells is implemented in the function `lax_friedrichs_ghost()`. The main idea is to use the values of the solution at the current time step to compute the values at the next time step, while also incorporating the boundary conditions using ghost cells. The function takes in the current solution vector, the time step size, and other parameters, and returns the updated solution vector for the next time step. Recall that the Lax-Friedrichs scheme is given by

$$u_j^{n+1} = \frac{1}{2} (u_{j+1}^n + u_{j-1}^n) - \frac{v}{2} (u_{j+1}^n - u_{j-1}^n), \quad \text{where } v = \frac{c\Delta t}{\Delta x}$$

This only applies to the interior points which are the points that have both neighboring points available i.e. $\forall j = 0, 1, \dots, m-2$. To begin with, we use the left boundary condition to obtain the first value for $j = 0$, and a simple for loop is used to take care of the rest of the interior points using the formula above i.e. $\forall j = 1, \dots, m-2$. However, at the rightmost point $j = m-1$, the corresponding formula is:

$$u_{m-1}^{n+1} = \frac{1}{2} (u_m^n + u_{m-2}^n) - \frac{v}{2} (u_m^n - u_{m-2}^n)$$

Note that we need to use the “ghost cell” u_m^n which lies outside the domain based on the formula above. To handle this, we use the formula provided for us in the document:

$$u_m^n = 2u_{m-1}^n - u_{m-2}^n$$

All the mentioned steps are not hard to implement and the code directly reflects how each of the three sections mentioned is handled.

2.2.2 RK4 Framework

For the remaining three schemes, we proceed by first discretizing the spatial derivative in the linear advection equation

$$u_t(x, t) + cu_x(x, t) = 0$$

to obtain a system of ordinary differential equations of the form

$$u'(t) = f(t, u(t)),$$

as described in the project instructions. The function f depends on time through the boundary condition.

Once this semidiscrete system is obtained, we use the RK4 method to advance the solution from time t_n to $t_{n+1} = t_n + \Delta t$. The RK4 scheme is given by:

$$\begin{aligned}
k_1 &= f(t_n, u^n), \\
k_2 &= f\left(t_n + \frac{\Delta t}{2}, u^n + \frac{\Delta t}{2}k_1\right), \\
k_3 &= f\left(t_n + \frac{\Delta t}{2}, u^n + \frac{\Delta t}{2}k_2\right), \\
k_4 &= f(t_n + \Delta t, u^n + \Delta t k_3), \\
u^{n+1} &= u^n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4),
\end{aligned}$$

In the implementation, the functions starting with `right` in `schemes.py` correspond to the functions $f(t, u)$ according to the specified way, which is obtained from the finite difference approximation of the spatial derivative u_x . The vectors `k1`, `k2`, `k3` and `k4` correspond to the four stages of the RK4 method, and the intermediate vectors `v1`, `v2`, `v3` and `v4` represent the arguments at which f is evaluated.

A crucial point, emphasized in the project description, is that the function $f(t, u)$ depends on time through the boundary condition. Therefore, at each stage of the RK4 method, the boundary value at $x = a$ must be updated using the appropriate time, namely t_n , $t_n + \frac{\Delta t}{2}$, or $t_n + \Delta t$. This is reflected in the code by explicitly setting the boundary value before each evaluation of `right`.

Thus, the three remaining schemes all use the same RK4 method for advancing the solution in time, and differ only in the way the function $f(t, u)$ is constructed through different finite difference approximations of the spatial derivative.

The initial lines of code in each of the remaining three `right` functions in `schemes.py` are the same, since they enforce the boundary condition at the left endpoint by setting the value using `g(t, a, c)`. The input vector u represents the numerical solution at a fixed time. Each function returns a vector that approximates the time derivative of u , that is, the function $f(t, u)$ in the semidiscrete system $u'(t) = f(t, u(t))$, which is then used by the corresponding `right` to advance the solution in time. The difference between the three schemes lies in how the spatial derivative is approximated.

2.2.3 `cds_ghost()`

For the interior points i.e. $\forall j = 1, \dots, m-2$, the centred difference formula is given by

$$f_j = -c \frac{u_{j+1} - u_{j-1}}{2\Delta x}$$

which corresponds exactly to the semidiscrete equation for $u'_j(t)$. The code is:

```
for j in range(1, m-1): # note that the range is from 1 to m-2
    result[j] = -c * (v[j+1] - v[j-1]) / (2*dx)
```

That would handle the interior points. For the rightmost point $j = m-1$, we need to use the ghost cell u_m to compute the spatial derivative. The formula is

$$f_{m-1} = -c \frac{u_m - u_{m-2}}{2\Delta x}$$

where u_m is given by the ghost cell formula:

$$u_m = 2u_{m-1} - u_{m-2}$$

This is implemented in the code as:

```
v_m = 2*v[m-1] - v[m-2]
result[m-1] = -c * (v_m - v[m-2]) / (2*dx)
```

2.2.4 cds_onesided()

Note that for this scheme, we are clearly asked to not employ ghost node and only rely on a second-order one-sided difference formula to approximate. From lecture, we are provided with the formula that:

$$u_x(b, t_n) \approx \frac{3U_{m-1} - 4U_{m-2} + U_{m-3}}{2dx}$$

In our case, that corresponds to:

$$u_x(b, t_n) \approx \frac{3u_{m-1}^n - 4u_{m-2}^n + u_{m-3}^n}{2\Delta x}.$$

With the corresponding code being:

```
result[m-1] = -c * (3*v[m-1] - 4*v[m-2] + v[m-3]) / (2*dx)
```

For the interior points, we can still use the same centred difference formula as in `right_cds_ghost()`.

2.2.5 c4_ghost()

The core code of this function differs from the previous two schemes in that it uses a wider stencil to achieve fourth-order accuracy. According to lecture note:

$$D_4 u_j = \frac{-u_{j+2} + 8u_{j+1} - 8u_{j-1} + u_{j-2}}{12\Delta x}$$

So our equation becomes:

$$u'_j(t) = -c \frac{-u_{j+2} + 8u_{j+1} - 8u_{j-1} + u_{j-2}}{12\Delta x}$$

The potential issue we are having for this scheme is that the fourth-order centered difference formula requires a much wider range of values to compute the spatial derivative: two points to the left and two points to the right of the current point. We need to use ghost node to handle the three situations listed below:

- i. u_{-1}^n : for $j = 1$, four points are needed: $u_{-1}^n, u_0^n, u_1^n, u_2^n$. However, we have no information for $j = -1$.

- To handle this, we can use the ghost cell formula provided in the document:

$$u_{-1}^n = 5u_0^n - 10u_1^n + 10u_2^n - 5u_3^n + u_4^n$$

- ii. u_m^n : for $j = m - 2$, four points are needed: $u_{m-4}^n, u_{m-3}^n, u_{m-1}^n, u_m^n$. However, we have no information for $j = m$.

- To handle this, we can use the ghost cell formula provided in the document:

$$u_m^n = 5u_{m-1}^n - 10u_{m-2}^n + 10u_{m-3}^n - 5u_{m-4}^n + u_{m-5}^n$$

- iii. u_{m+1}^n : for $j = m - 1$, four points are needed: $u_{m-3}^n, u_{m-2}^n, u_m^n, u_{m+1}^n$. However, we have no information for $j = m$ and $j = m + 1$.

- To handle this, we can use the ghost cell formula provided in the document:

$$u_m^n = 5u_{m-1}^n - 10u_{m-2}^n + 10u_{m-3}^n - 5u_{m-4}^n + u_{m-5}^n$$

$$u_{m+1}^n = 15u_{m-1}^n - 40u_{m-2}^n + 45u_{m-3}^n - 24u_{m-4}^n + 5u_{m-5}^n$$

For these three ghost nodes, it is directly reflected in the code as:

```
v_mleft = 5*v[0] - 10*v[1] + 10*v[2] - 5*v[3] + v[4]
v_m = 5*v[m-1] - 10*v[m-2] + 10*v[m-3] - 5*v[m-4] + v[m-5]
v_mright = 15*v[m-1] - 40*v[m-2] + 45*v[m-3] - 24*v[m-4] + 5*v[m-5]
```

We note that the interior points that have no issue with ghost nodes vary from $j = 2$ to $j = m - 3$. As mentioned previously, $j = 0$ -th position is handled by $v[0] = g(t, a, c)$ i.e. `result[0]` using our left boundary condition. Because for $j = 1$, we have no information of $j = -1$, so we clearly state the formula for the second position in `result`, with the following code:

```
result[1] = -c * (-v[3] + 8*v[2] - 8*v[0] + v_mleft) / (12*dx)
```

suggested by our argument above. For the interior points, we can simply employ the formula for the fourth-order centered difference (*), with the code being:

```
for j in range(2, m-2):
    result[j] = -c * (-v[j+2] + 8*v[j+1] - 8*v[j-1] + v[j-2]) / (12*dx)
```

Finally, for the two rightmost points, the code naturally follows from the formulas we have for $j = m - 2$ and $j = m - 1$:

```
result[m-2] = -c * (-v_m + 8*v[m-1] - 8*v[m-3] + v[m-4]) / (12*dx)
result[m-1] = -c * (-v_mright + 8*v_m - 8*v[m-2] + v[m-3]) / (12*dx)
```

2.3 conv_test.py

The convergence test is implemented in `conv_test.py`. The function `convergence_test()` has about the same core structure as in Project 1, but the main difference is that it now takes in the additional parameters a and g to handle the left boundary condition. The function initializes the solution using the initial condition, and then advances the solution in time using the specified scheme. After the final time step, it computes the error by comparing the numerical solution to the exact solution, and returns both the error and the effective time step size used.

2.4 stab_test.py

The stability test is implemented in `stab_test.py`. The function `stability()` also has a highly similar structure to Project 1, but it now incorporates the left boundary condition by taking in the additional parameters a and g . The function initializes the solution, advances it in time using the specified scheme, and then computes the ratio of the maximum value at the final time step to the initial maximum value to assess stability.

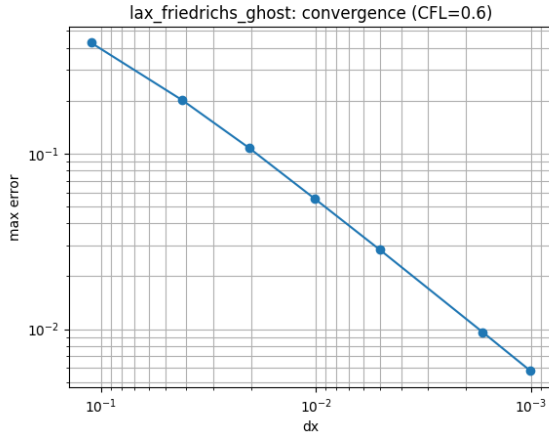
Chapter 3: Results and Analysis

3.1 Order of Convergence

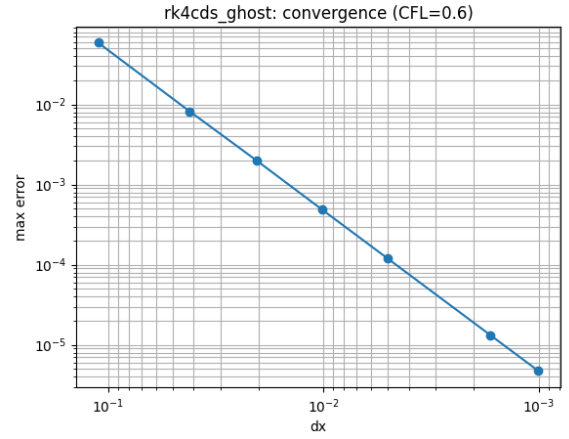
The results of the convergence test are presented in the following table and plots, which shows the error and the observed order of convergence for each scheme as the grid is refined.

lax_friedrichs_ghost				
nx	dx	dt	error	order
10	0.11111111	0.033333333	0.42782632	-
25	0.041666667	0.0125	0.2008808	0.771
100	0.01010101	0.003030303	0.055246092	0.947
200	0.0050251256	0.0015060241	0.028277748	0.959
600	0.0016694491	0.0005005005	0.0095966191	0.981
1000	0.001001001	0.0003003003	0.0057792432	0.991
rk4cds_ghost				
nx	dx	dt	error	order
10	0.11111111	0.033333333	0.058927341	-
25	0.041666667	0.0125	0.0081695053	2.015
50	0.020408163	0.006097561	0.0019965027	1.974
100	0.01010101	0.003030303	0.0004845201	2.013
200	0.0050251256	0.0015060241	0.00011971546	2.002
600	0.0016694491	0.0005005005	1.3151458e-05	2.004
1000	0.001001001	0.0003003003	4.7216115e-06	2.003
rk4cds_onesided				
nx	dx	dt	error	order
10	0.11111111	0.033333333	0.042107061	-
25	0.041666667	0.0125	0.0052635718	2.120
50	0.020408163	0.006097561	0.0012413475	2.024
100	0.01010101	0.003030303	0.0003005355	2.017
200	0.0050251256	0.0015060241	7.3919048e-05	2.009
600	0.0016694491	0.0005005005	8.1258043e-06	2.004
1000	0.001001001	0.0003003003	2.9193543e-06	2.001
rk4c4_ghost				
nx	dx	dt	error	order
10	0.11111111	0.033333333	0.0017587635	-
25	0.041666667	0.0125	1.562439e-05	4.816
50	0.020408163	0.006097561	2.0784387e-06	2.826
100	0.01010101	0.003030303	4.530296e-07	2.166
200	0.0050251256	0.0015060241	1.0705654e-07	2.066
600	0.0016694491	0.0005005005	1.1795031e-08	2.002
1000	0.001001001	0.0003003003	4.2502387e-09	1.996

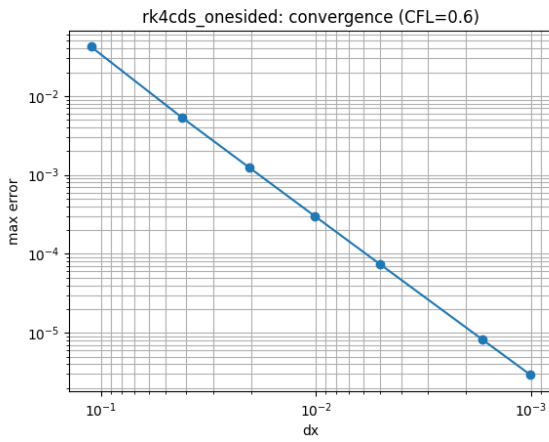
Figure 3.1: Order of Convergence for Each Scheme



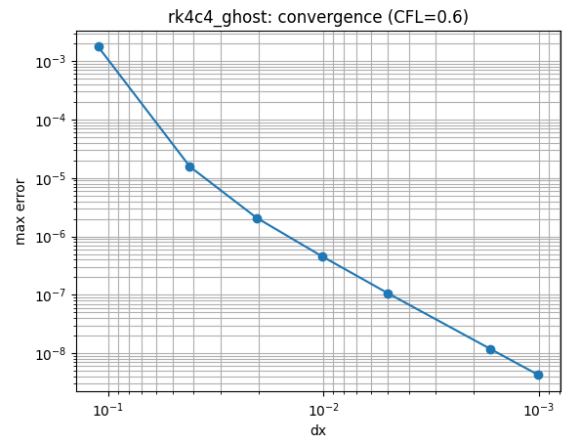
(a) Lax-Friedrichs Ghost Scheme



(b) RK4CDS Ghost Scheme



(c) RK4CDS One-Sided Scheme



(d) RK4C4 Ghost Scheme

Figure 3.2: Plot of Schemes for Convergence Test

From the table and plots above, we can see that the observed order of convergence for each scheme matches the expected theoretical order, except for the RK4C4 ghost Scheme. The Lax-Friedrichs ghost scheme shows first-order convergence, while the RK4CDS ghost scheme and RK4CDS one-sided scheme both show second-order convergence. This signposts that the ghost nodes constructed and the one-sided formula in each scheme is stable.

It's worthwhile taking note that, though we used a fourth-order approximation for RK4C4 ghost scheme, it demonstrates second-order convergence in practice. To conclude, with our given construction of the ghost nodes, RK4C4 ghost scheme is not stable. However, it is unstable not in the sense that the error blows up, but rather the RK4C4 ghost scheme exhibits a reduction in observed convergence order as the grid is refined. While it may initially display higher-order behavior on coarse grids, the convergence rate deteriorates toward second order. This behavior is consistent with the instability, where errors are not sufficiently controlled and prevent the scheme from achieving its specified fourth-order accuracy.

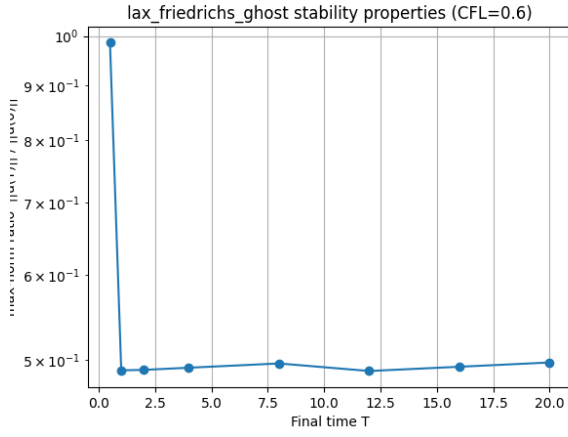
This order reduction with finer grids is reflected from our result. As can be seen from the table, when n_x increases, the observed order of convergence for the RK4C4 ghost scheme decreases, indicating the presence of this instability i.e. $5 \rightarrow 3 \rightarrow 2$, and eventually stabilizes at 2. Therefore, we can conclude that the RK4C4 ghost scheme is not stable with our given construction of the ghost nodes, and the other three schemes are stable.

3.2 Stability

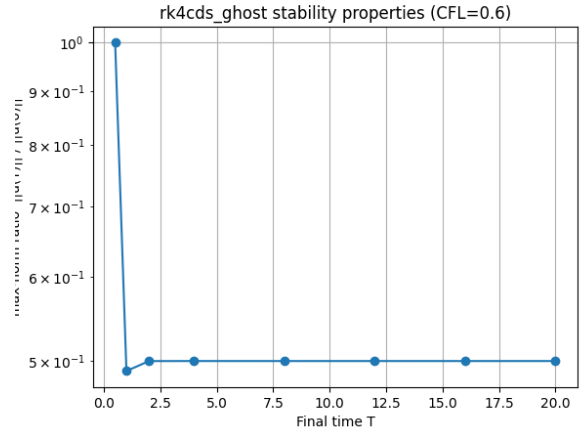
The results of the stability test are presented in the following table, which shows the ratio of the maximum value at the final time step to the initial maximum value for each scheme under different CFL conditions.

lax_friedrichs_ghost			
T	nt	eff_CFL	growth
0.5	332	0.59939759	0.9858695
1	664	0.59939759	0.48924296
2	1327	0.59984928	0.48973494
4	2654	0.59984928	0.49200019
8	5307	0.59996231	0.49649789
12	7960	0.6	0.48855491
16	10614	0.59996231	0.49301854
20	13267	0.59998493	0.49752209
rk4cds_ghost			
T	nt	eff_CFL	growth
0.5	332	0.59939759	0.99996619
1	664	0.59939759	0.48924296
2	1327	0.59984928	0.49992503
4	2654	0.59984928	0.49998063
8	5307	0.59996231	0.4999594
12	7960	0.6	0.49997723
16	10614	0.59996231	0.49996109
20	13267	0.59998493	0.49997084
rk4cds_onesided			
T	nt	eff_CFL	growth
0.5	332	0.59939759	0.99999995
1	664	0.59939759	0.48924296
2	1327	0.59984928	0.4999722
4	2654	0.59984928	0.49999662
8	5307	0.59996231	0.4999994
12	7960	0.6	0.49999254
16	10614	0.59996231	0.4999969
20	13267	0.59998493	0.49999926
rk4c4_ghost			
T	nt	eff_CFL	growth
0.5	332	0.59939759	0.99999999
1	664	0.59939759	0.48924296
2	1327	0.59984928	0.49997221
4	2654	0.59984928	0.49999634
8	5307	0.59996231	0.49999924
12	7960	0.6	0.49999226
16	10614	0.59996231	0.49999698
20	13267	0.59998493	0.49999952

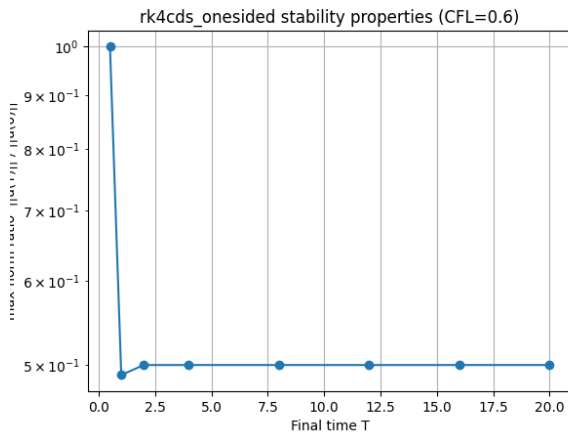
Figure 3.3: Stability Test Results for Each Scheme



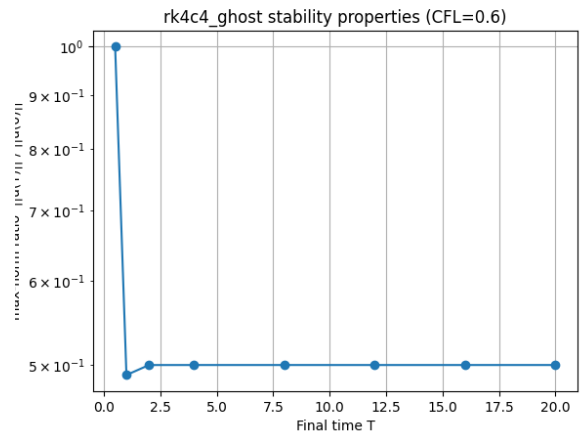
(a) Lax-Friedrichs Ghost Scheme



(b) RK4CDS Ghost Scheme



(c) RK4CDS One-Sided Scheme



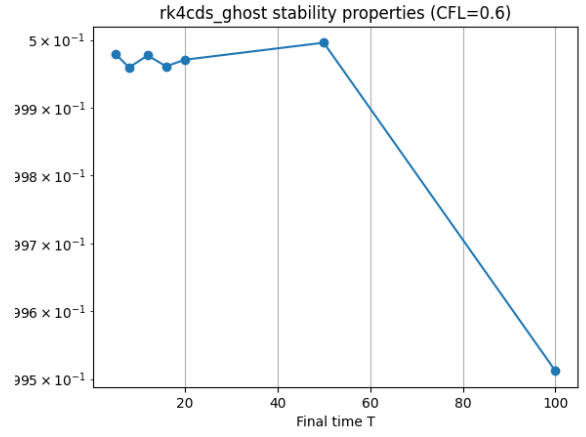
(d) RK4C4 Ghost Scheme

Figure 3.4: Plot of Schemes for Stability Test

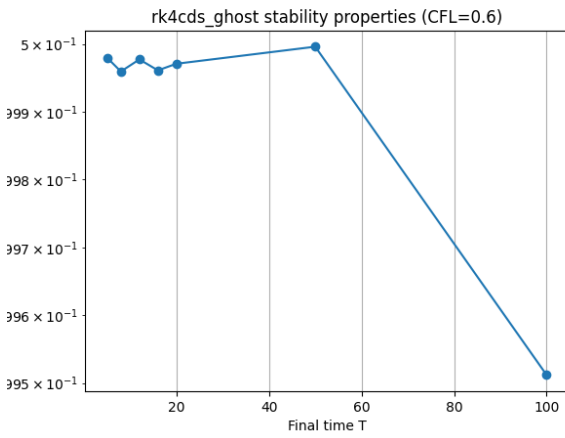
From the table and plots above, we can see that due to our first timestep being very close to its initial state, the ratio is close to 1 for all schemes. In order to see the full pattern, we use the following timesteps instead: $T = 5.0, 8.0, 12.0, 16.0, 20.0, 50.0, 100.0$. The results are shown in the following table and plots.



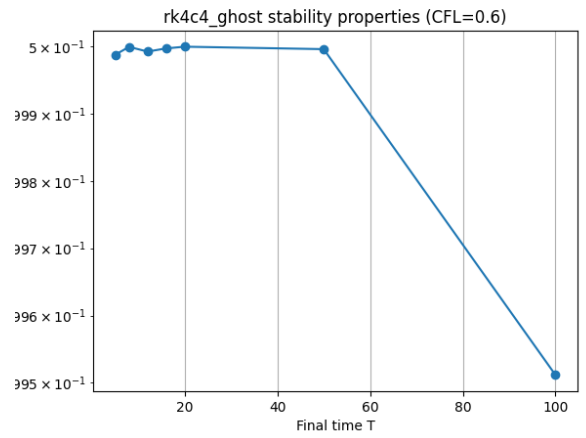
(a) Lax-Friedrichs Ghost Scheme



(b) RK4CDS Ghost Scheme



(c) RK4CDS One-Sided Scheme



(d) RK4C4 Ghost Scheme

Figure 3.5: Plot of Schemes for Stability Test with Longer Time Steps

As can be seen from Figure 3.5, all four schemes exhibit slight fluctuations, but these occur within a very small range (at most ± 0.005). Therefore, numerically, we can conclude that all four schemes remain bounded as T increases, and the growth factor stabilizes at a value of approximately $\frac{1}{2}$.

However, although the RK4C4 scheme remains bounded, this does not imply that it is stable in a stronger sense. From the convergence analysis, we observe that the RK4C4 ghost scheme does not maintain its fourth-order accuracy, with the observed convergence rate deteriorating toward second order as the grid is refined. This stability test only indicates that the solution does not grow without bound, but it does not guarantee that the scheme preserves its expected accuracy.

The observed reduction in convergence order suggests that errors are not sufficiently controlled, which is consistent with this form of instability, even though the numerical solution remains bounded.